

Vision Transformer

An Image is Worth 16x16 Words:
Transformers for Image Recognition at Scale

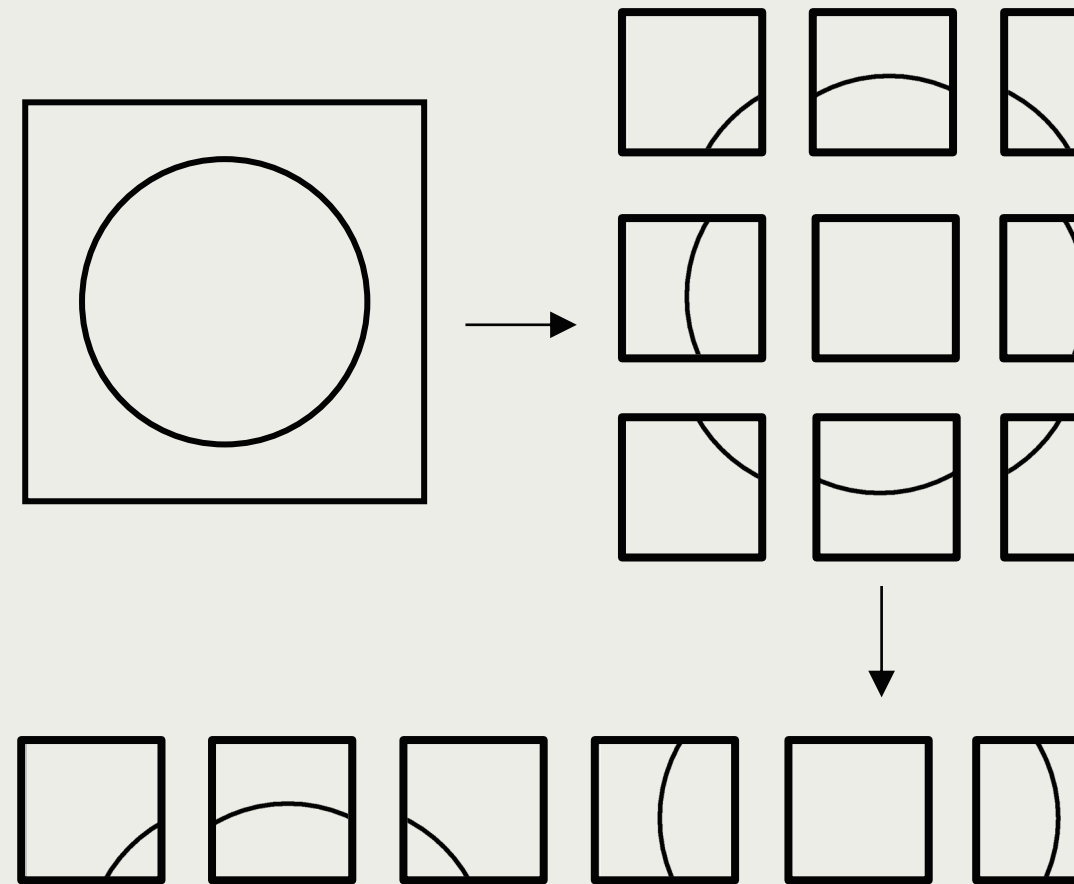


Image Processing Team | Detection

Byunghyun Kim, Chanhyuk Lee, Eungi Hong, Jongsik Ahn,
Hyeonjin Kim, Jaewan Park, Chungchun Hyun, Seonok Kim

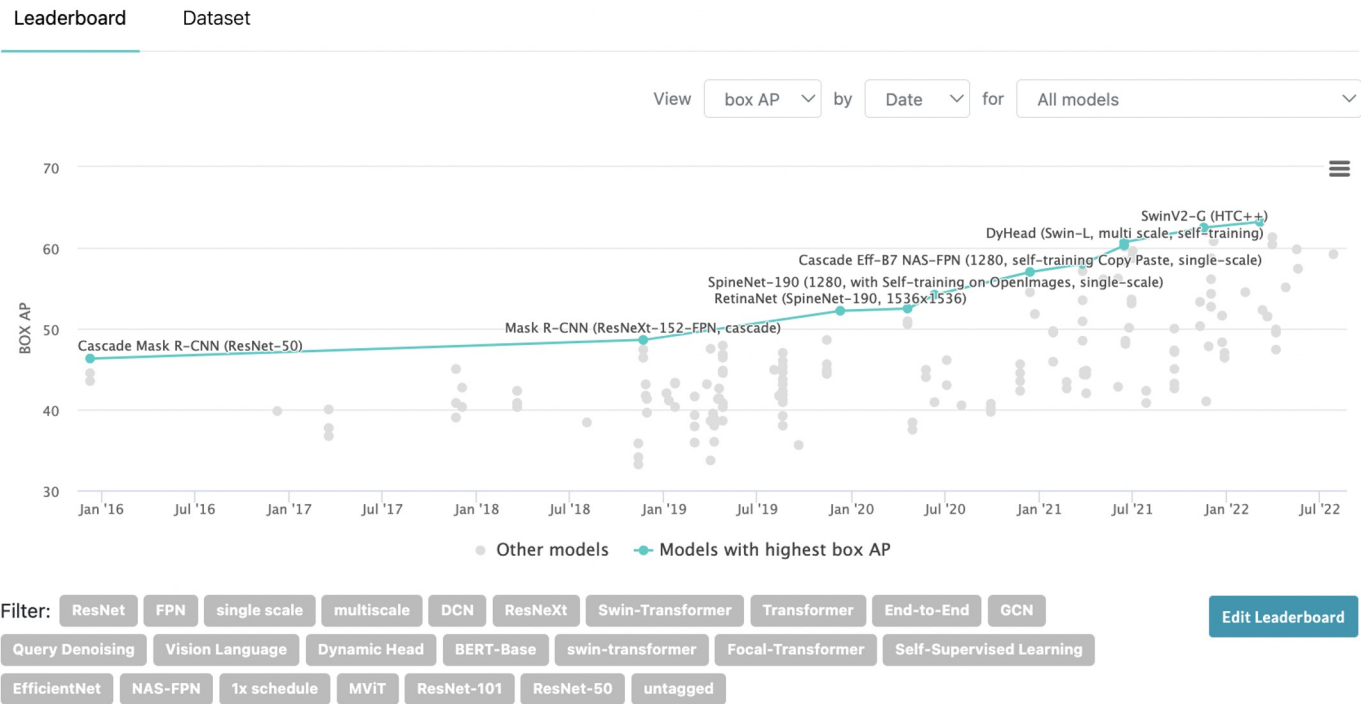
ICLR 2021

Proposed by Alexey et al.
Google Research, Brain Team

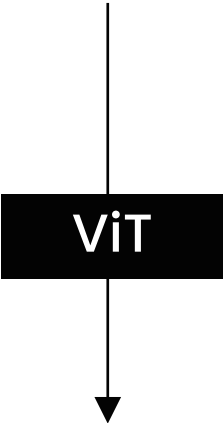
Introduction

Transformers for Vision Trasks

Object Detection on COCO minival



CNN-based Models



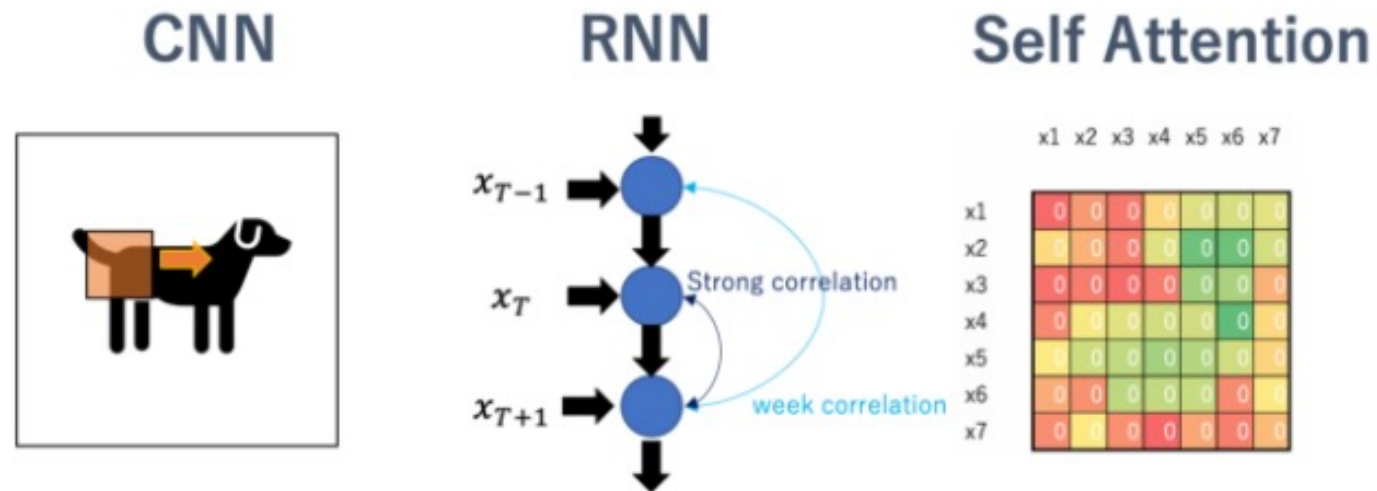
Transformer-based Models

Attention and Self-Attention

- Attention is a mechanism that was developed to improve the performance of the Encoder-Decoder.
- Self-attention aggregates information from all the other words, generating a new representation per word informed by the entire context.

Inductive Bias

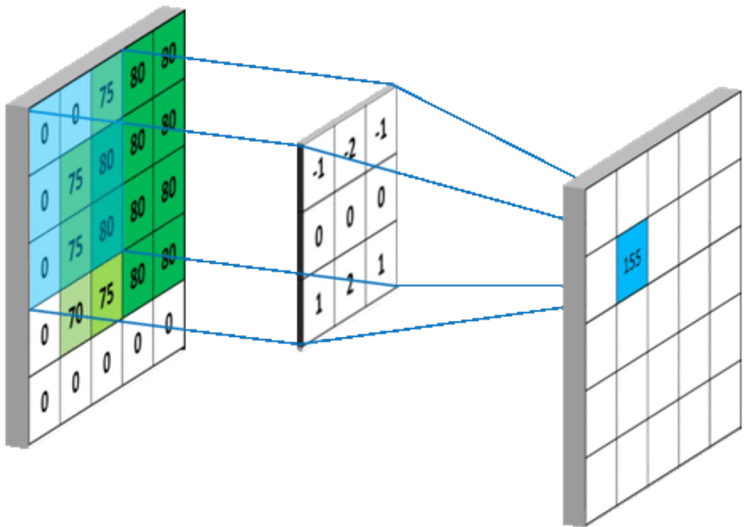
- Inductive Bias is the assumptions made by the model to learn the target function and to generalize beyond training data.



(Left) CNN, which has a strong inductive bias that the information is locally aggregated. (center) RNN, which has a strong inductive bias in that it is strongly correlated with the previous time (right) Self-Attention, which has a relatively weak inductive bias because it only correlates all features.

CNN vs Transformer

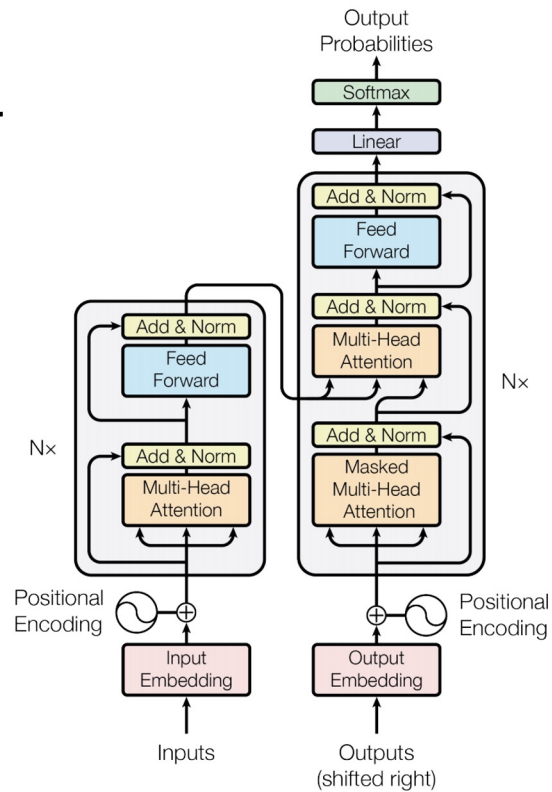
- CNNs have two biases that are directly intrinsic to the very functioning of the architecture.
- These biases are not present in the Transformers architecture and so they need more data to fully understand the problem.



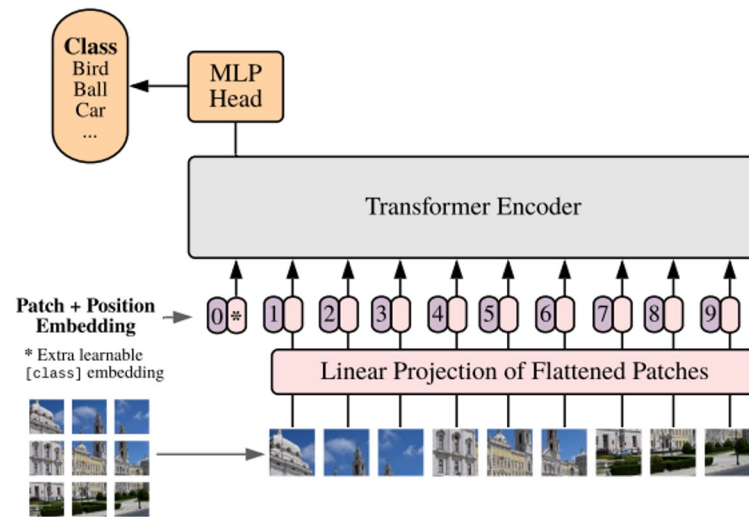
Motivation

- The reliance on CNNs is not necessary and a pure transformer applied directly to sequences of image patches can perform very well on image classification tasks.

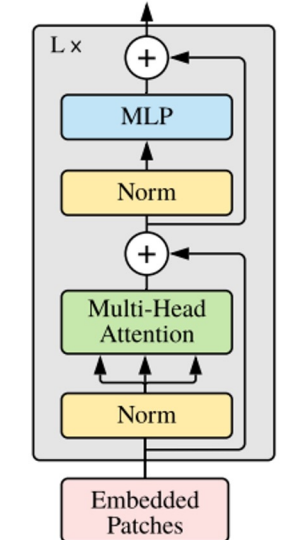
Transformer for NLP



Vision Transformer



Transformer Encoder



Method

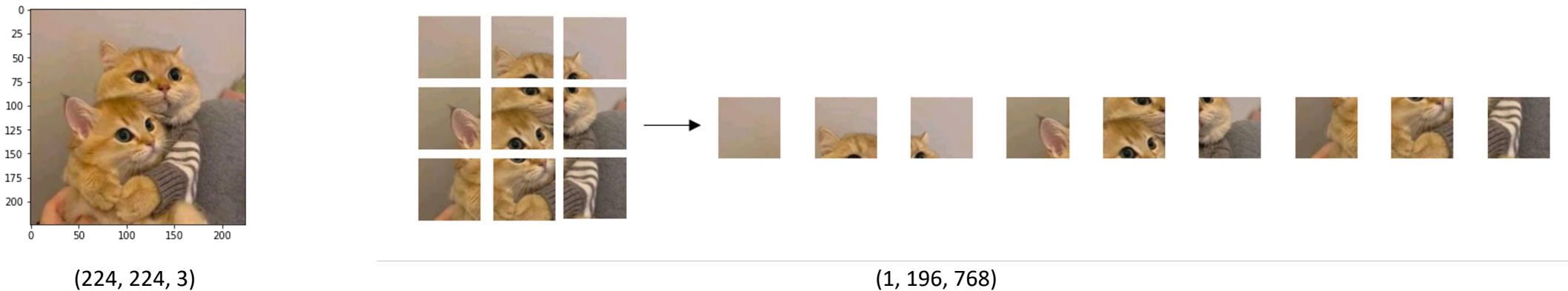
Vison Transformer (ViT)

- Vision Transformer is a model for image classification that views images as sequences of smaller patches.
- Vision Transformers work by splitting an image into a sequence of smaller patches, use those as input to a standard Transformer encoder.



Patch Embedding

- The standard transformer receives as input a 1D sequence of token embeddings.
- To handle 2D images, we reshape the image X into a sequence of flattened 2D patches.



```
x = convert_tensor(img).unsqueeze(0) # add batch dim on the first axis
# torch.Size([1, 3, 224, 224])

patch_size = 16 # 16 pixels
patches = rearrange(x, 'b c (h s1) (w s2) -> b (h w) (s1 s2 c)', s1=patch_size, s2=patch_size)
# 1, 3, (14*16), (14*16) -> 1, (14*14), (16*16*3)

# torch.Size([1, 196, 768])
```

$$x \in \mathbb{R}^{H \times W \times C} \xrightarrow{\text{reshape}} x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$$

H, W : the resolution of the original image

C : the number of channels

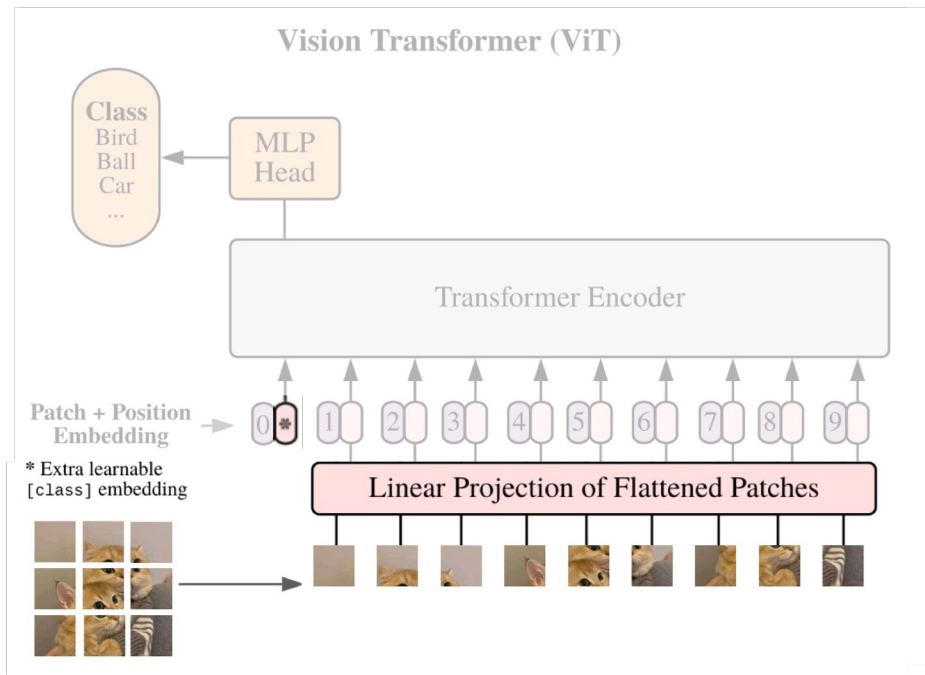
(P, P) : the resolution of each image patch

N : the resulting number of patches HW/P^2

D : the constant latent vector size

Linear Projection and CLS token

- The patches can be projected using a normal linear layer.
- A CLS token is added to serve as representation of an entire image, which can be used for classification.



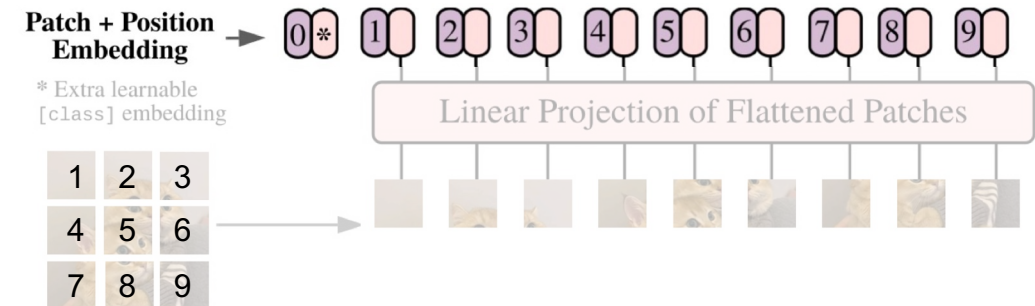
```
def forward(self, x: Tensor) -> Tensor:
    b, _, _, _ = x.shape
    x = self.proj(x)
    cls_tokens = repeat(self.cls_token, '() n e -> b n e', b=b)
    # prepend the cls token to the input
    x = torch.cat([cls_tokens, x], dim=1)
    return x
```

```
# torch.Size([1, 197, 768])
```

Position Embedding

- Since the model has no idea about the original position of the patches, we need to pass this spatial information.
- The position embedding is a tensor of shape $(N_PATCHES + 1 \text{ (token)}, EMBED_SIZE)$ that is added to the projected patches.

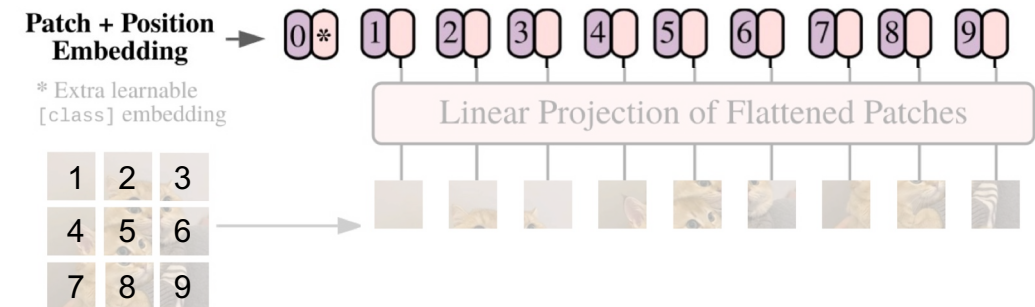
```
def forward(self, x: Tensor) -> Tensor:
    b, _, _, _ = x.shape
    x = self.proj(x)
    cls_tokens = repeat(self.cls_token, '() n e -> b n e', b=b)
    # prepend the cls token to the input
    x = torch.cat([cls_tokens, x], dim=1)
    # add position embedding
    x += self.positions
    return x
```



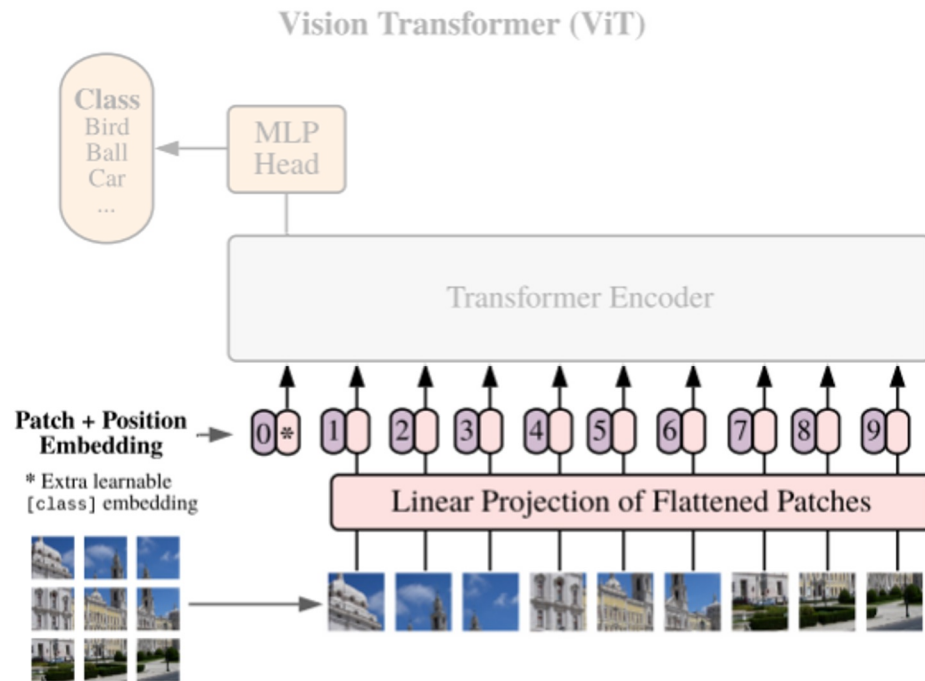
Position Embedding

- Since the model has no idea about the original position of the patches, we need to pass this spatial information.
- The position embedding is a tensor of shape $(N_PATCHES + 1 \text{ (token)}, EMBED_SIZE)$ that is added to the projected patches.

```
def forward(self, x: Tensor) -> Tensor:
    b, _, _, _ = x.shape
    x = self.proj(x)
    cls_tokens = repeat(self.cls_token, '() n e -> b n e', b=b)
    # prepend the cls token to the input
    x = torch.cat([cls_tokens, x], dim=1)
    # add position embedding
    x += self.positions
    return x
```



Patch Embedding



```
class PatchEmbedding(nn.Module):
    def __init__(self, in_channels: int = 3, patch_size: int = 16, emb_size: int = 768, img_size: int = 224):
        self.patch_size = patch_size
        super().__init__()
        self.proj = nn.Sequential(
            nn.Conv2d(in_channels, emb_size, kernel_size=patch_size, stride=patch_size),
            Rearrange('b e (h) (w) -> b (h w) e'),
        )

        self.cls_token = nn.Parameter(torch.randn(1, 1, emb_size))
        self.positions = nn.Parameter(torch.randn((img_size // patch_size) ** 2 + 1, emb_size))

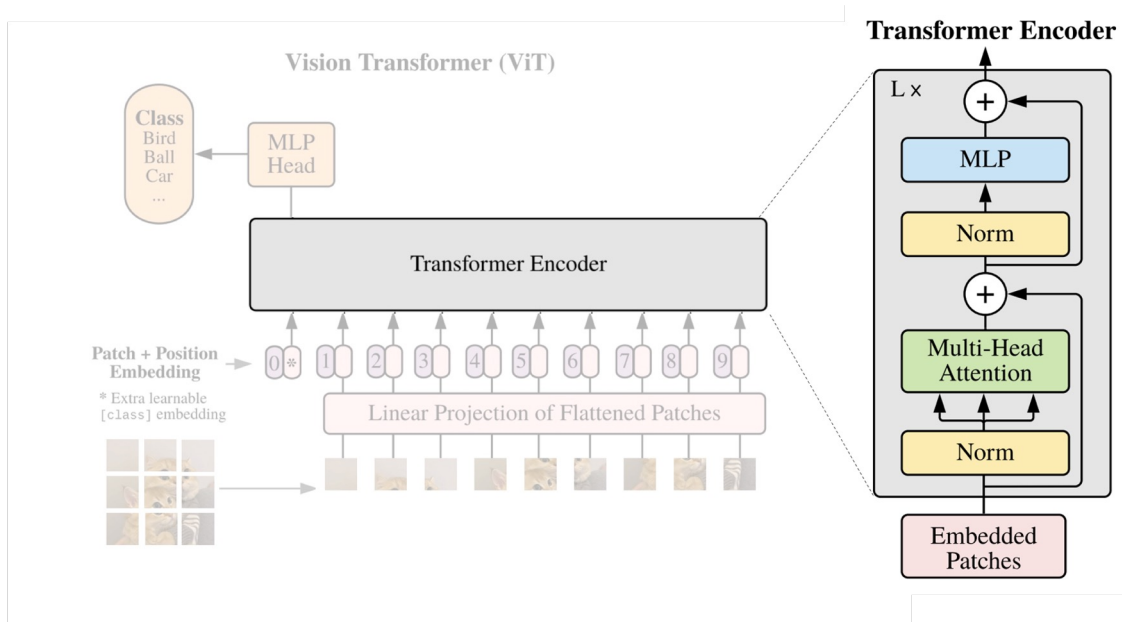
    def forward(self, x: Tensor) -> Tensor:
```

```
        b, _, _, _ = x.shape
        x = self.proj(x)
        cls_tokens = repeat(self.cls_token, '() n e -> b n e', b=b)
        # prepend the cls token to the input
        x = torch.cat([cls_tokens, x], dim=1)
        # add position embedding
        x += self.positions
        return x
```

```
PatchEmbedding()(x).shape
# torch.Size([1, 197, 768])
```

Transformer Encoder

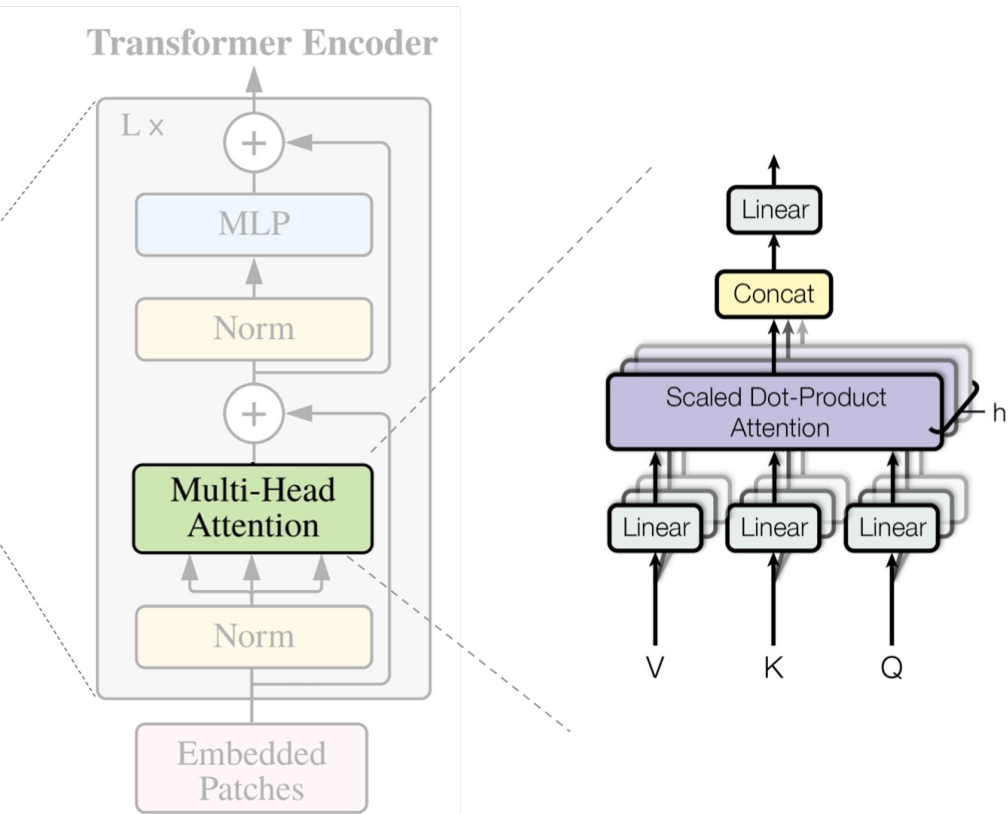
- Only the encoder is used for ViT.
- Multi-Head attention, residual addition and MLP are used for the encoder.



```
ResidualAdd(nn.Sequential(
    nn.LayerNorm(emb_size),
    MultiHeadAttention(emb_size, **kwargs),
    nn.Dropout(drop_p)
)),
ResidualAdd(nn.Sequential(
    nn.LayerNorm(emb_size),
    FeedForwardBlock(
        emb_size, expansion=forward_expansion, drop_p=forward
    ),
    nn.Dropout(drop_p)
))
```


Multi-Head Attention

- The multi-head attention splits the computation across n heads with smaller size.
- We need to perform matrix multiplication between queries and keys in order to compute the attention matrix.



```
# define keys, queries, values
emb_size = 512
num_heads = 8
keys = nn.Linear(emb_size, emb_size)
queries = nn.Linear(emb_size, emb_size)
values = nn.Linear(emb_size, emb_size)

# split keys, queries and values in num_heads
queries = rearrange(queries(x), "b n (h d) -> b h n d", h=num_heads)
keys = rearrange(keys(x), "b n (h d) -> b h n d", h=num_heads)
values = rearrange(values(x), "b n (h d) -> b h n d", h=num_heads)
```

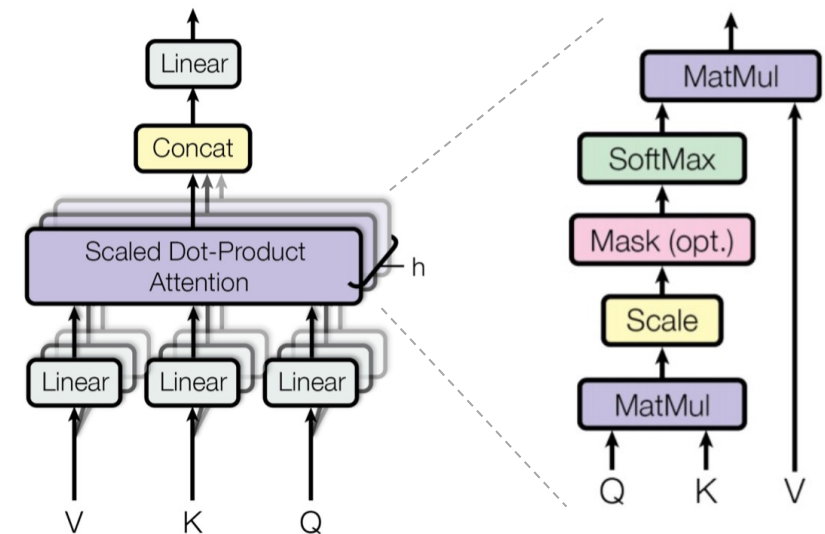

Multi-Head Attention

- The softmax of the resulting vector is divided by a scaling factor which is based on the size of the embedding.

```
# sum up over the last axis
energy = torch.einsum('bhqd, bhkd -> bhqk', queries, keys)
# energy = batch, num_heads, query_len, key_len

# divided by a scaling factor
scaling = self.emb_size ** (1/2)
att = F.softmax(energy, dim=-1) / scaling
att = self.att_drop(att)

# sum up over the third axis
out = torch.einsum('bhal, bhlv -> bhav ', att, values)
out = rearrange(out, "b h n d -> b n (h d)")
out = self.projection(out)
```



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

MLP (Multilayer Perceptron)

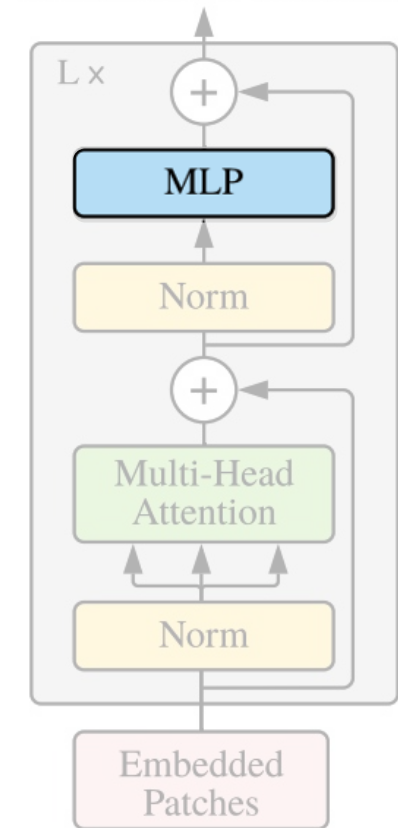
- The output of attention is passed to a fully connected layer composed of two layers.
- The layers upsample inputs by an expansion factor.

```
nn.Linear(emb_size, expansion * emb_size),
nn.GELU(),
nn.Dropout(drop_p),
nn.Linear(expansion * emb_size, emb_size)
```

The MLP contains two layers with a GELU non-linearity.

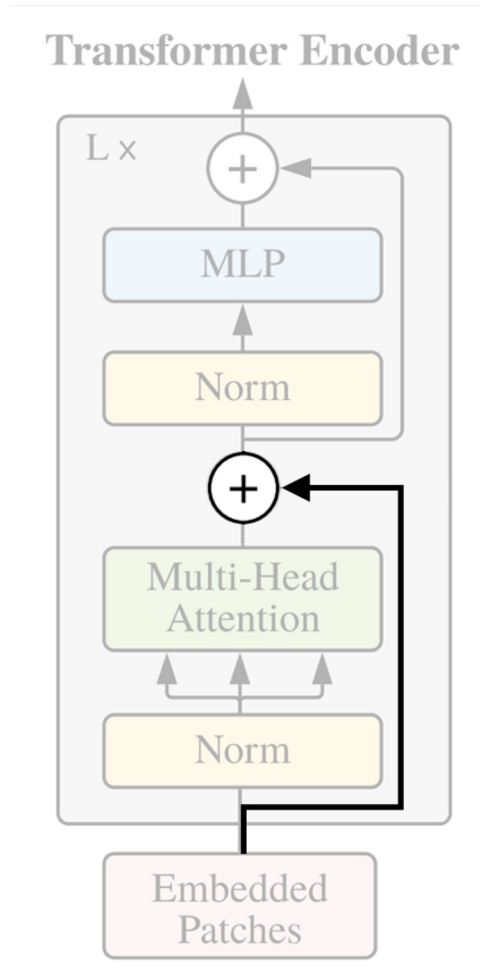
$$\begin{aligned} \mathbf{z}_0 &= [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, & \mathbf{E} &\in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D} \\ \mathbf{z}'_\ell &= \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, & \ell &= 1 \dots L \\ \mathbf{z}_\ell &= \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, & \ell &= 1 \dots L \\ \mathbf{y} &= \text{LN}(\mathbf{z}_L^0) \end{aligned}$$

Transformer Encoder



Residuals

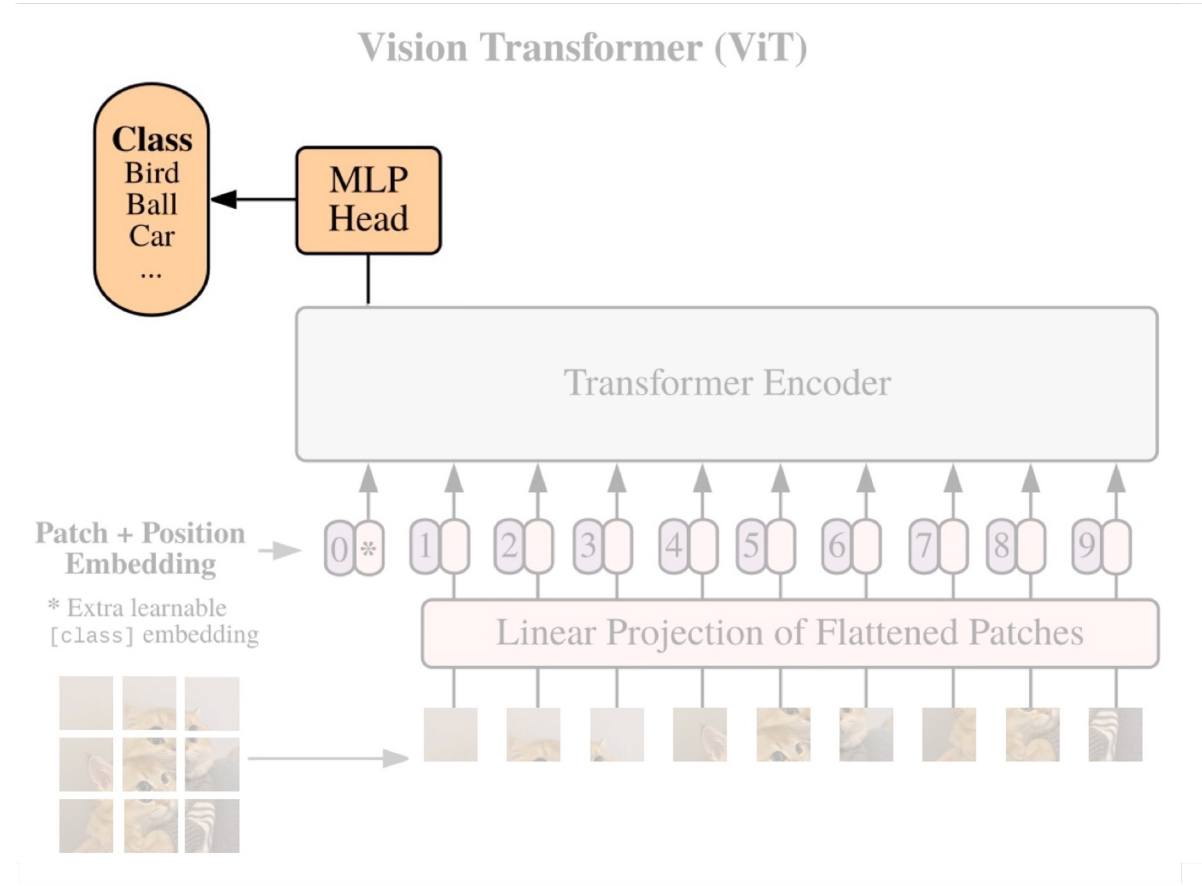
- Layernorm (LN) is applied before every block, and residual connections after every block.



```
class ResidualAdd(nn.Module):  
    def __init__(self, fn):  
        super().__init__()  
        self.fn = fn  
  
    def forward(self, x, **kwargs):  
        res = x  
        x = self.fn(x, **kwargs)  
        x += res  
        return x
```

MLP Head

- The last fully connect layer gives the class probability.
- It performs a basic mean over the whole sequence.

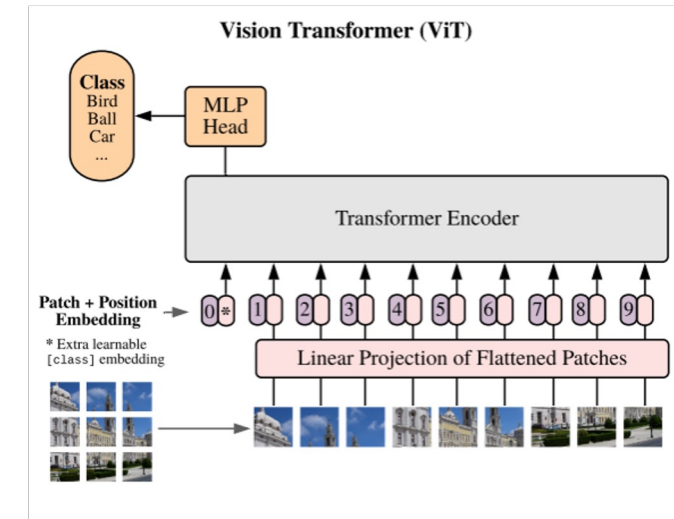


```
Reduce('b n e -> b e', reduction='mean'),
nn.LayerNorm(emb_size),
nn.Linear(emb_size, n_classes)
```

ViT

- PatchEmbedding, TransformerEncoder and ClassificationHead can be composed in order to create the ViT architecture.

```
class ViT(nn.Sequential):
    def __init__(self,
                  in_channels: int = 3,
                  patch_size: int = 16,
                  emb_size: int = 768,
                  img_size: int = 224,
                  depth: int = 12,
                  n_classes: int = 1000,
                  **kwargs):
        super().__init__(
            PatchEmbedding(in_channels, patch_size, emb_size, img_size),
            TransformerEncoder(depth, emb_size=emb_size, **kwargs),
            ClassificationHead(emb_size, n_classes)
        )
```



```
=====
Total params: 86,415,592
Trainable params: 86,415,592
Non-trainable params: 0
-----
```

```
Input size (MB): 0.57
Forward/backward pass size (MB): 364.33
Params size (MB): 329.65
Estimated Total Size (MB): 694.56
-----
```

Question 🤔

Experiments

Model Variants & Benchmarks

- The “Base” and “Large” models are directly adopted from BERT and the larger “Huge” model added.
- Vision Transformer models pre-trained on the JFT-300M dataset outperform ResNet-based baselines on all datasets.

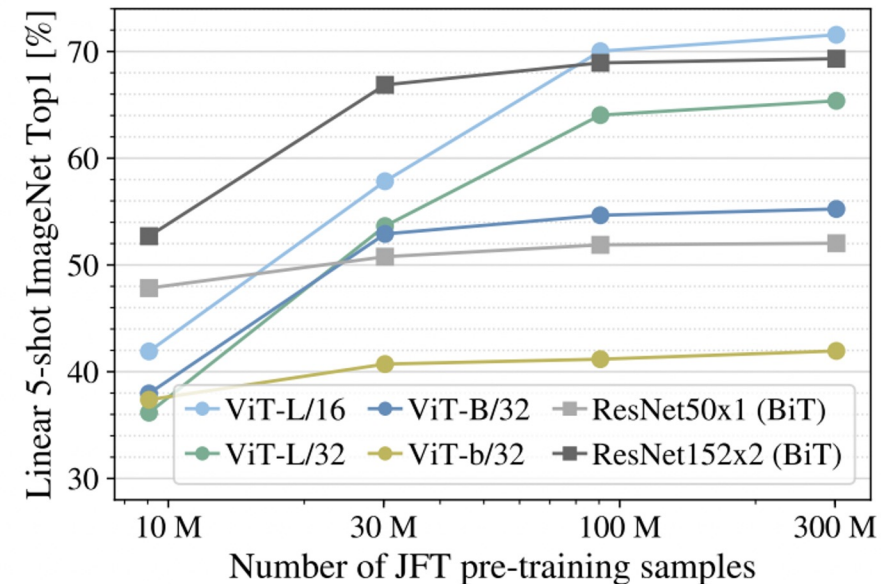
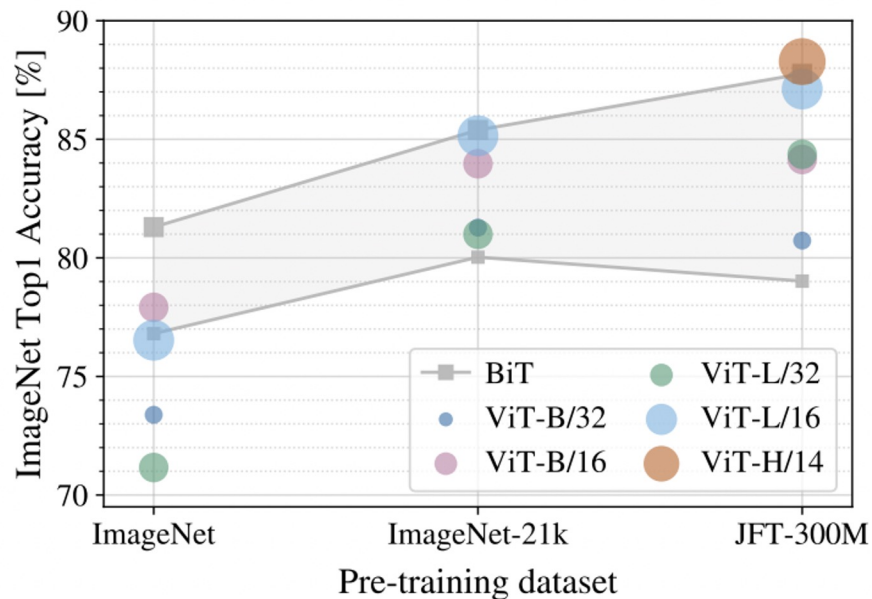
Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

Table 1: Details of Vision Transformer model variants.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21K (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

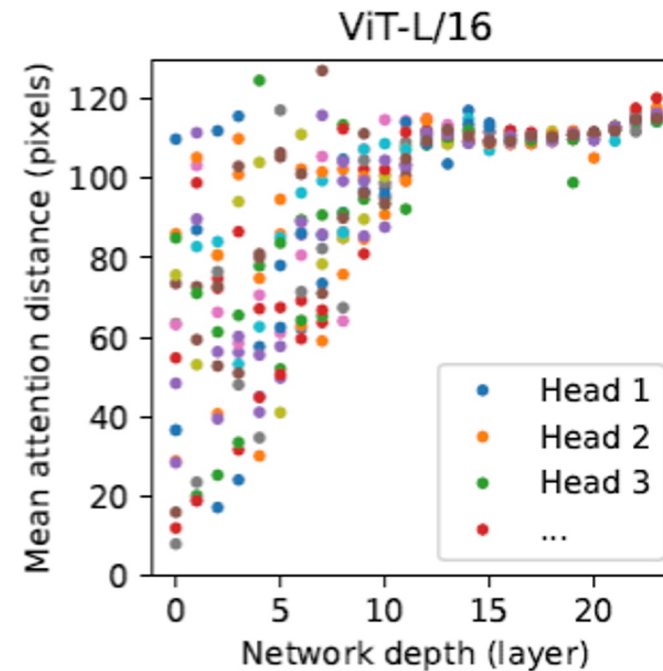
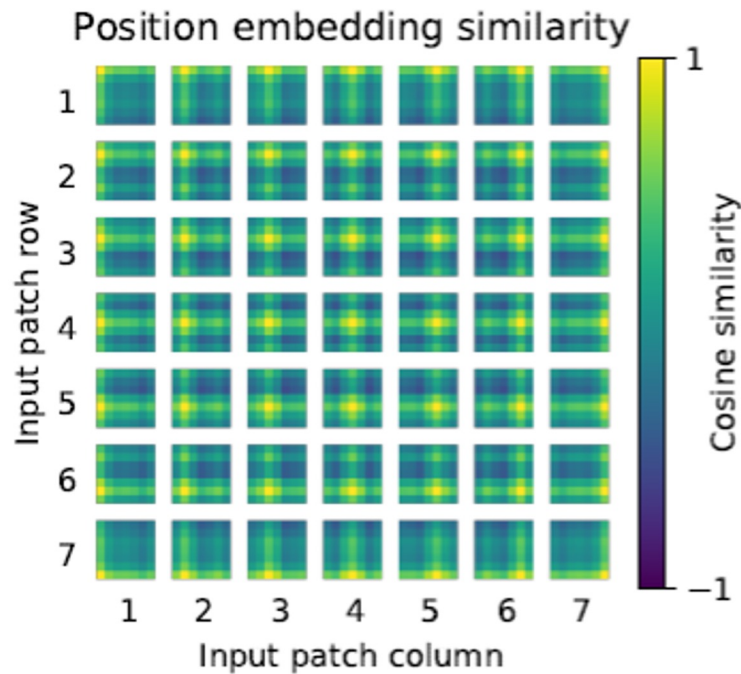
Transfer Performance & Linear Few-shot Evaluation

- While large ViT models perform better when pre-trained on larger datasets, they perform worse than BiT ResNets (shaded area) when pre-trained on small datasets.
- ResNets perform better with smaller pre-training datasets but plateau sooner than ViT, which performs better with larger pre-training.



Position Embedding Similarity & Size of Attended Area

- Tiles on the left shows the cosine similarity between the position embedding of the patch with the indicated row and column and the position embeddings of all other patches.
- Each dot on the right shows the mean attention distance across images for one of 16 heads at one layer.

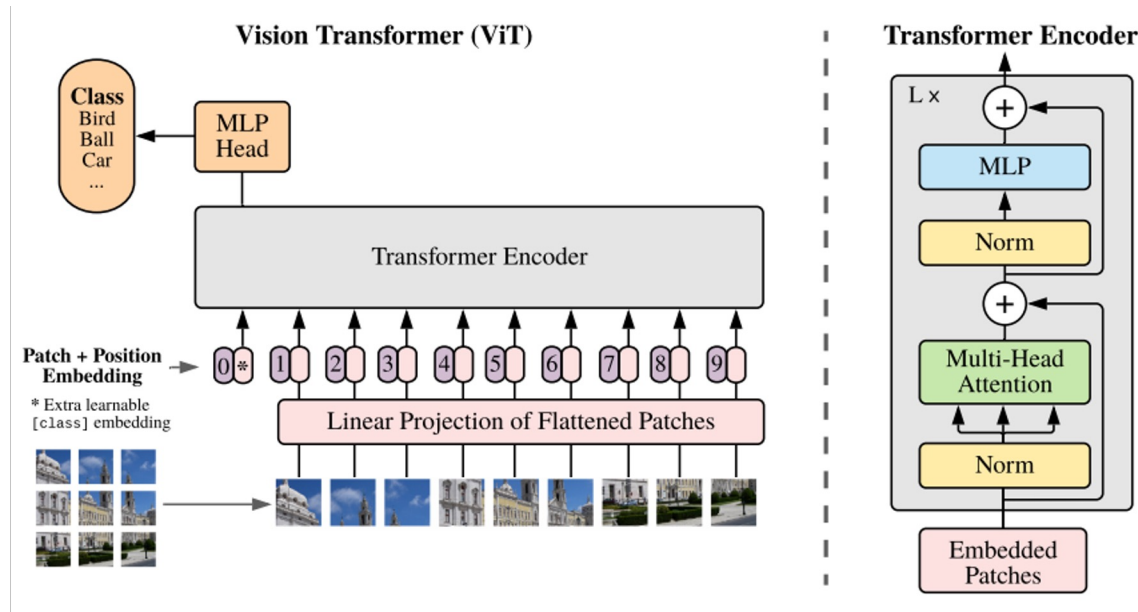


Question 🤔

Conclusion

Conclusion

- Unlike prior works using self-attention in computer vision, the authors interpret an image as a sequence of patches and process it by a standard Transformer encoder as used in NLP.
- Vision Transformer (ViT) attains excellent results compared to state-of-the-art convolutional networks while requiring fewer computational resources to train.



Sources

Paper

- Dosovitskiy, Alexey, et al. "An image is worth 16x16 words: Transformers for image recognition at scale." International Conference on Representation Learning (ICLR 2021)
- Ashish Vaswani, et al. Attention is All You Need (NIPS 2017)

Blog

- Towards Data Science (towardsdatascience.com/implementing-visual-transformer-in-pytorch-184f9f16f632)

Seonok Kim

E-mail: seonokkim@gmail.com
Blog: medium.com/@seonokkim